Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Isabella Trevino	itrev@udel.edu
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

I want to create an app where users build their own vehicle “garage”. They could design the appearances of these cars and do body work on them.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

This concept has been implemented multiple times as a web-based application. For example, the Dream Car Garage allows you to look up real cars and put them in your garage bays. There are also websites with more customization options, like 3D Tuning, where you can choose a model, change the color of the car down to specific panels, choose between interior and exterior, etc.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

Because of time, software, and experience limitations, my car design would be much simpler. Graphics would likely be more “8-Bit” than fully rendered 3D models. I hope I can

make the simplicity of my application an attractive rather than off-putting feature. I intend for it to be more of a background app, something you can divert your attention to when you need a break from whatever you are working on. By necessity (drafter websites having no memory) there are no stakes to what you create; they will be gone the next time you open the app. It's about the process of doing it, not the result.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

The app will receive user input like make, model, color, modifications, and convert it into a pixel-formatted representation of a car that they can view on the webpage and further modify.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

Preferences for the vehicles they are designing, like color, model, make, etc.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

Preferences will be compared against a database of available customization options, for example, a file of strings representing colors, and create features accordingly through arrays and looping. The results will be output to the user on the website page as a simple graphic

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

A simple car graphic that adopts the preferences they define in the app.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Cars that have been previously defined, different preferences as the user goes through the process of making a new car.

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Need: body color, body style, brand Ignore: complex body styles/packages, small aesthetic details (headlights, tire wall colors, etc.)

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

Taking user input and producing a car that remembers and keeps all preferences.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I would like to manually code most of this project, but I will likely need the help of an LLM to get it fully operational. I will try to dedicate 1 hr each day of the week to developing the program.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

Become comfortable with designing something meant for ANYONE to interact with, not just myself. I want it to be creative and unexpected but not so niche that nobody cares.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

ChatGPT: I want to create a web app using Drafter that can take user input and return a pixel art car that has all the user's specified preferences (like color, model, etc.), and to store that car in state until the server restarts/web page is refreshed. What kind of customizations would be the most practical to implement

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

ChatGPT suggested selecting a few specific customization metrics, like color, a choice of 3 or so body styles, a wheel size, etc. I would start with making dictionaries of colors and body styles. Car styles would be arrays of strings, and each string would represent a color. When passed through a function, the strings are translated to colors and a pixel car is output. The AI did not mention any specific benchmark examples of existing apps.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

Wouldn't wheel size be pretty difficult, since I would have to move the car upward or downward to accommodate for the new height? How can I make this easier to code?

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

I have decided that I need to restrict the amount of pixels I use for generating graphics or else this project will become overwhelming very quickly. Although I would prefer a greater selection of body styles, it would benefit me most to narrow it down to just a few choices. It is a lot more simplistic that I had imagined at first, but more realistic.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

Office hours, Friday open classes, allowed LLMs.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

I will first respond to the LLM with my issue and see how it reacts. Hopefully as we continue conversating, it will remember this issue and avoid suggestions that cause it again. For simple issues, I can try simply unit testing my car generation functions, scanning the code for syntax issues, etc. Thonny's debugging feature can help me step through my code to find the problem. If this still doesn't help I can always go to TA office hours for in person assistance.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Me: I am trying to design a web app where users design a car garage. Users choose a make and model for their car, plug in additional things like color and modifications, and the website generates a pixel art car matching their input. I want the graphics to be very simple,

like 8-bit art. I want to be able to write the code myself, so I was wondering if there are any python libraries or techniques I can use to generate this pixel art in a way that is not too complicated for me to hand write, like with arrays.
ChatGPT: I think this is a really good fit for a CS1 project, especially because you can generate pixel art using simple Python data structures instead of AI image generation or complicated graphics software.

For your Drafter web app, I'd recommend building a small library of pixel-art car templates and then changing their colors and adding modifications based on the user's choices. That gives users a garage of customized cars while keeping the code understandable enough to write and debug yourself.

The key design decision is to build one recognizable car shape first, then make it customizable, rather than trying to generate every real-world car model from scratch.

1. The simplest technique: represent a car as a 2D array

Imagine that your car is made of square pixels. Each pixel is represented by a character or number in a Python list.

For example, a tiny car might be represented like this:

```
None
car = [    "....RR....",    "...RRRR...",    "..RRRRRR..",
"RRRRRRRRRR",    "RRRRRRRRRR",    ".WW....WW."]
```

Each character represents a color or a transparent space:

- `R` = red body
- `W` = black or dark wheel (you could rename this code)
- `.` = transparent background

You could then write a function that reads the array and draws a colored square for each character. The result would be a small pixel-art sprite.

The benefit is that you can literally edit the car's appearance by changing the array. Adding a row, changing a letter, or moving a pixel changes the car's shape.

2. Three ways to create your pixel-art cars

Pixel art red car. Vector 8 bit game web icon isolated on white background.

Option A — Character arrays (my recommendation)

Best for learning

Store the car as rows of characters in a Python list. Use a dictionary to map characters to colors, then draw each square. You control the shape, body color, windows, wheels, and details.

Pixel Art Vehicle Coloring Pages - 40 Printable Drawings (Free PDF)

Option B — Draw rectangles and shapes

Instead of specifying every pixel, use coordinates to draw the car body, roof, windows, and wheels as rectangles. This is easier to customize by car type, although it takes a little more coordinate math.

vector pixel art cartoon

Option C — Layered templates

Draw a base car, then overlay separate layers for paint, wheels, spoilers, headlights, and other modifications. This is probably the best long-term structure for a garage builder.

You can even combine these approaches: use a character array for the base shape and separate arrays or drawing functions for modifications.

3. Which Python libraries should you consider?

Library or technique	How it helps	Fit for Drafter
Plain Python lists and dictionaries	Stores pixel layouts and color palettes	Excellent
Browser SVG	Draws colored rectangles from generated markup	Worth exploring
Browser Canvas API	Draws individual pixels and rectangles on a drawing surface	Good, but requires browser interop

Pillow	Creates and edits raster images	Possible, but displaying the generated image in a browser needs an extra step

Pillow is a well-established Python image library with drawing functions for shapes and pixels.

Pillow (PIL Fork) 12.3.0 documentation
Drafter also supports a Canvas component, although its documentation notes that drawing on it requires interacting with the browser's JavaScript API.
Drafter Documentation

My recommendation: Start with plain Python arrays and render them using SVG or Canvas. You do not need to introduce a graphics library just to draw a small pixel-art car.

One important detail: Drafter runs your Python in the browser using Pyodide, so a third-party library needs to be compatible with that environment.

Drafter Documentation+1
A library working on your regular computer does not automatically mean it will work in Drafter.

4. How to make the car respond to user choices

This is where your app becomes interesting. You can separate the car's shape from its properties.

For example, your data might look like this:

```
None
car_shapes = {    "sports_car": [        "....RR....",
"...RRRR...",        "..RRRRRR..",        "RRRRRRRRRR",
"RRRRRRRRRR",        ".WW....WW."    ]}colors = {    "R": "red",
"W": "black",    ".": None}
```

This is just a miniature example to demonstrate the concept, not a finished car sprite.

Next, the user chooses a paint color. Rather than changing every `R` in the array, you can use a variable for the car's paint color:

```
None
paint_color = "blue"wheel_color = "black"
```

Your rendering function can interpret the body pixels using `paint_color`, wheel pixels using `wheel_color`, and transparent pixels as empty space.

Then the same base template can generate a red, blue, green, or black car.

How modifications could work

User selection	How to implement it
Paint color	Change a color variable
Wheel style	Select between wheel templates
Spoiler	Add a small array or draw extra rectangles
Body style	Choose a different base template
Windows	Draw a separate window layer
Headlights	Change a few pixels or their colors

For make and model selection, I'd initially map each option to one of a few simplified body templates. For instance, several sports cars might share a sports-car silhouette, while a pickup truck gets a longer body and a truck cab.

This is much more manageable than trying to reproduce the exact geometry of every real vehicle. The app would generate a stylized interpretation of the selected car, rather than a

photorealistic or technically accurate model.

5. What would the rendering logic look like?

Here's the basic algorithm you would eventually implement:

1. Collect user selections

Car style, paint, wheels, modifications

2. Select the car template

Load the correct 2D array and modification layers

3. Render each pixel

Loop through rows and columns, translating symbols into colors and squares

4. Display the finished car

Show the customized sprite in the user's virtual garage

In Python, the central logic is just nested loops:

```
None
for y, row in enumerate(car):    for x, pixel in enumerate(row):
# Decide which color this pixel should use.        # Draw a
square at position (x, y).        pass
```

You would replace `pass` with the drawing operation appropriate to your chosen renderer. If using Canvas, each pixel can be drawn as a filled rectangle; if using SVG, you can generate a rectangle element for each non-transparent pixel.

6. Keep the first version achievable

For a three-week introductory project, I'd build it in stages:

- Stage 1: Draw one car with a hard-coded array.
- Stage 2: Let users change the paint color.
- Stage 3: Add two or three different body templates.
- Stage 4: Add one or two modifications, such as a spoiler or different wheels.
- Stage 5: Build the garage interface where users can view their customized cars.

If time allows, you could also save configurations in browser local storage. You would be

saving the car's settings and template choice, rather than saving a generated image or building a backend database.

I would postpone automatic generation of arbitrary car models, realistic modifications, and a large catalog of makes until the basic renderer works.

7. One last design decision

Before you start coding, I'd choose how to draw the squares. Since you're using Drafter, I'd lean toward SVG rectangles generated from Python, provided your course's Drafter setup supports the SVG component as expected. It may avoid some of the complexity of drawing directly through JavaScript's Canvas API.

The official Drafter Canvas documentation explains the alternative approach, including its browser interop requirements.

The one question I'd settle before planning the code is:

Do you want the cars to look like recognizable real-world makes and models, or is it okay if they are simplified car types (sports car, sedan, pickup, hatchback) with the make and model serving mainly as labels?

That choice determines how many templates you'll need and how complicated the project will become.

Me: The drafter website gives this example but it doesnt work when I try to run it

```
from drafter import *

from dataclasses import dataclass




@dataclass

class State:

    pass



@route
```

```
def index(state: State) -> Page:
    return Page(state, [
        Header("Doodle"),
        Canvas("doodle", width=300, height=120),
        "\n",
        Button("Draw a sunset", "draw")
    ])



@route
def draw(state: State) -> Page:
    import js
    canvas = js.document.getElementById("doodle")
    if canvas is not None:
        brush = canvas.getContext("2d")
        brush.fillStyle = "coral"
        brush.fillRect(0, 0, 300, 120)
        brush.fillStyle = "gold"
        brush.beginPath()
        brush.arc(150, 120, 40, 3.14, 0)
        brush.fill()
    return Update(state)
```

start_server(State())

ChatGPT: This example is trying to do something useful for your pixel-art car project: it draws directly onto a browser HTML canvas using Python and JavaScript together.

The important part is that the drawing happens inside the `draw` route, after you click the button. The `Canvas` component creates the drawing surface, and `js.document.getElementById("doodle")` tries to find it in the webpage.

There are a few possible reasons it might not work, and the exact error message will help us identify which one applies.

First, try this small diagnostic change

The documentation's example is valid according to the current Drafter reference, so I wouldn't rewrite everything yet.

Drafter Documentation

Temporarily change the `draw` route to this:

```
None
@routedef draw(state: State) -> Page:    import js    canvas = js.document.getElementById("doodle")    print("Canvas found:", canvas is not None)    if canvas is not None:        brush = canvas.getContext("2d")        brush.fillStyle = "coral"        brush.fillRect(0, 0, 300, 120)    return Update(state)
```

This removes the sunset drawing so we can focus on whether the canvas is being found. Look at the browser console or Drafter's debugging output for the print message.

- If it prints `Canvas found: True` but nothing draws, we should investigate the canvas drawing operation.
- If it prints `Canvas found: False`, the problem is locating the canvas element.
- If you get an error involving `js`, the Python code may be running somewhere other than the browser's Pyodide environment. The `import js` belongs inside the route, as in the original example.
  Drafter Documentation

Could you tell me what happens when you run it? Specifically, do you get an error message when starting the app, an error after clicking Draw a sunset, or does the button click simply do nothing? If there's an error, paste the exact text here. That will let us fix the actual

problem rather than guessing.

Me: i just get an orange box

ChatGPT: Good news — the orange box means the canvas is working! The issue is most likely the direction in which the example draws the semicircle.

The original code has:

```
None
brush.arc(150, 120, 40, 3.14, 0)
```

Canvas draws arcs clockwise by default. With these angles, the arc goes along the lower half of the circle, which falls outside your canvas because its center is at the bottom edge (`y = 120`).

Fix: add `True` to draw the upper half

Replace the sunset drawing code with this:

```
None
@routedef draw(state: State) -> Page:    import js    canvas = js.document.getElementById("doodle")    if canvas is not None:
brush = canvas.getContext("2d")        # Coral background
brush.fillStyle = "coral"        brush.fillRect(0, 0, 300, 120)
# Gold semicircle        brush.fillStyle = "gold"
brush.beginPath()        brush.arc(150, 120, 40, 3.14, 0, True)
brush.fill()    return Update(state)
```

The extra `True` tells Canvas to draw the arc counterclockwise, making the semicircle appear above the bottom edge.

You should now see a gold semicircle at the bottom center of the orange box.

Why this matters for your car project: you've already confirmed that Python can draw on the browser's canvas. The same `fillRect()` function can draw individual colored squares from your pixel-art arrays — so this is a promising approach for your garage app!

Me: now the website does not pull up at all. the REPL is frozen as

>>>

ChatGPT: Don't worry — this is probably recoverable! Since the website stopped loading after the code change and the REPL is sitting at `>>>`, let's get the app running again before changing anything else.

1. Try restarting the app

1. Press Ctrl+C in the REPL to interrupt anything that might still be running.
2. If you get the `>>>` prompt back, that means the Python interpreter is ready for another command.
3. Run your original, complete program again, including the final line:

```
None
```

4.
```
start_server(State())
```

5. Wait for the server to start, then refresh the Drafter webpage.

If Ctrl+C does nothing, try the REPL's Restart or Reset option, if your environment provides one.

2. Use this safer version of the drawing code

One correction to my previous advice: the arc's direction may affect the drawing, but it should not prevent the entire website from starting. Let's remove the arc temporarily and test the simplest working drawing.

Keep your imports, `State`, and `index` route as they were. Replace your `draw` route with this:

```
None
@routedef draw(state: State) -> Page:    import js    canvas = js.document.getElementById("doodle")    if canvas is not None:
brush = canvas.getContext("2d")        brush.fillStyle = "coral"
brush.fillRect(0, 0, 300, 120)        brush.fillStyle = "gold"
brush.fillRect(130, 70, 40, 40)    return Update(state)
```

This should produce an orange canvas with a gold square when you click the button. Once that works, we can add the semicircle back.

3. One important question

When you say the REPL is frozen at `>>>`, which of these describes what you see?

What happens after you run your code?

I see `>>>` and can type commands, but the website won't load.

I see `>>>`, but I can't type anything or get a response.

I see an error message before or after the prompt.

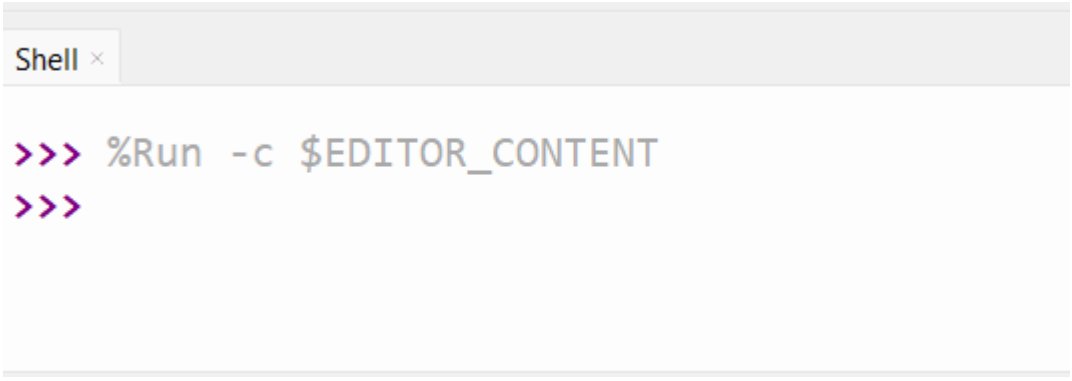
Where are you running Drafter?

In a browser-based course environment or online editor

Locally on my computer, such as VS Code or a terminal

Help me troubleshoot

The answers matter because a `>>>` prompt normally means Python is waiting for input, not necessarily that it is frozen. We can narrow this down without rebuilding your project.



Me:

it shows this and does nothing

ChatGPT: Thanks, this screenshot helps! The REPL is probably not frozen. The `>>>` prompt means Python is ready for another command. The `%Run -c $EDITOR_CONTENT` line indicates your editor tried to run your code, then returned to the prompt.

My suspicion is that the program is finishing without starting the Drafter server, or the code currently in the editor is incomplete.

Step 1: Check the very bottom of your code

Make sure the last line of your entire program is:

```
None
start_server(State())
```

It must not be indented inside a function. If that line is missing, the website won't start.

Step 2: Try this complete test program

To avoid chasing multiple problems, replace the contents of your editor temporarily with this entire program:

```
None
from drafter import *from dataclasses import
dataclass@dataclassclass State:    pass@routedef index(state:
State) -> Page:    return Page(state, [        Header("Doodle"),
Canvas("doodle", width=300, height=120),        Button("Draw",
"draw")    ])@routedef draw(state: State) -> Page:    import js
canvas = js.document.getElementById("doodle")    if canvas is not
None:        brush = canvas.getContext("2d")
brush.fillStyle = "coral"        brush.fillRect(0, 0, 300, 120)
return Update(state)start_server(State())
```

Run the entire file using your editor's Run button.

Step 3: Tell me what happens

- Does the website open again?
- Does the REPL return to `>>>` immediately?
- Does any error appear in the shell?

If it still returns immediately to `>>>`, please send me a screenshot of the code in your editor, including the bottom of the file. That will help me identify the problem without making you try random changes.

We can get this working before moving on to drawing your pixel-art cars.

Me: this is all the shell says, and the website is back to showing just an orange rectangle

```
20 def draw(state: State) -> Page:
21     import js
22
23     canvas = js.document.getElementById("doodle")
24     if canvas is not None:
25         brush = canvas.getContext("2d")
```

Shell

```
INFO:     Application startup complete.
INFO:     Drafter ready at http://localhost:8000
INFO:     ::1:58344 - "GET / HTTP/1.1" 200 OK
INFO:     ::1:60519 - "WebSocket /__drafter_ws" [accepte
d]
INFO:     connection open
```

ChatGPT: Great, this screenshot tells us something important: your Drafter server is running correctly! The shell says `Application startup complete` and `Drafter ready at http://localhost:8000`. You do not need to restart everything.

The remaining problem is just the drawing. The orange rectangle appears because that part works. Let's fix the semicircle.

1. Check the rest of your `draw` function

In your screenshot, the code cuts off at line 25, so I can't see whether the arc code below it is correct.

Make sure the entire function looks exactly like this:

None

```
@routedef draw(state: State) -> Page:    import js    canvas =
js.document.getElementById("doodle")    if canvas is not None:
brush = canvas.getContext("2d")        brush.fillStyle = "coral"
brush.fillRect(0, 0, 300, 120)        brush.fillStyle = "gold"
brush.beginPath()        brush.arc(150, 120, 40, 3.14, 0, True)
brush.fill()    return Update(state)
```

The important line is:

```
None
brush.arc(150, 120, 40, 3.14, 0, True)
```

The final `True` tells the browser to draw the arc counterclockwise, so the gold semicircle appears above the bottom edge of the canvas instead of below it, where it would be clipped.

2. Run it and click the button again

After replacing the function:

1. Run the entire program again.
2. Open or refresh the Drafter page.
3. Click Draw a sunset.

You should see the orange rectangle with a gold semicircle at the bottom center.

If you still see only orange, send me a screenshot showing the complete `draw` function, including the lines below 25. Your server is running, so we can focus on the few lines responsible for the drawing rather than changing anything else.

I can see the problem now! Your code is almost correct. The issue is the `True` at the end of your `brush.arc()` line. I gave you the wrong direction in my earlier instructions — sorry about that!

Fix one line

Change this:

```
None
brush.arc(150, 120, 40, 3.14, 0, True)
```

To this:

```
None
brush.arc(150, 120, 40, 3.14, 0, False)
```

Or simply remove the final argument:

```
None
brush.arc(150, 120, 40, 3.14, 0)
```

Why this works

Canvas uses `True` to draw the arc counterclockwise. With your circle centered at `(150, 120)`, that direction sends the arc below the bottom of your canvas, where it gets clipped.

Using `False` draws the upper semicircle, which should be visible inside the orange rectangle.

Now run your program again and click “Draw a sunset.” You should see a gold semicircle rising from the bottom center of the orange box. Your server and the rest of your code are fine!

Me: Can you show me how coding a simple car design from an array would work?

ChatGPT: You'd add `"G": "green"` to your dictionary, and then change the car array so that the body pixels use `G` instead of `R`.

For example, your color dictionary becomes:

```
None
colors = {    "R": "red",    "G": "green",    "B": "lightblue",
"W": "black"}
```

Then change your car array to this:

```
None
car = [    "....BBBB....",    "..BBBBBBBB..",    ".GGGGGGGGG.",
"GGGGGGGGGGG",    "GGGGGGGGGGG",    ".WW....WW..."]
```

That's it! Because your drawing code already looks up each character in `colors`, it will automatically draw the `G` pixels in green.

Want to let users choose the color?

You don't actually need a separate array for every color. You could use a variable:

```
None
paint_color = "green"colors = {    "R": paint_color,    "B":
"lightblue",    "W": "black"}
```

Now you can change just one line:

```
None
paint_color = "red"
```

or

```
None
paint_color = "green"
```

The car array stays exactly the same, and your drawing code doesn't need to change.

For your garage app, I'd recommend the second approach. It separates the car's shape from its paint color, which makes it much easier to let users customize cars through a Drafter dropdown menu.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.